Lecture 7: Lexical elements in C language: 17/6/2025

Lesson objectives:

By the end of this lesson the learner will be able to:

- Identify the valid character sets used to write a C program.
- Categorize the different types of tokens in C, including keywords, identifiers, constants, strings, and operators.
- Apply proper syntax rules to write valid C statements.
- Implement both single-line and multi-line comments correctly within a C program to improve code readability.

Lecture 7: Lexical elements in C language:

The lexical elements of a language are the words, punctuation, and grammar rules of the language.

a) Tokens : The basic unit of a program that convey a specific meaning to the compiler. There are five different types of tokens:
   - *Identifiers : For variables, functions and constants.*
   - *Keywords : Reserved words*
   - *Literals : Constants that have meaning/value*
   - *Operators : That define type of calculations*
   - *Punctuators: To punctuate statements.*

b) Character set: Set of alphabets, special characters and digits. ASCII xcters
   - Alphabets e.g. Aa,Bb,Cc,Dd,….
   - Digits e.g. 0,1,2,3,4,…
   - Special characters e.g. #,$,&,:
   - Whitespace e.g. tab space, new line, blank.

c) Comments: A brief description of the program/code. Preceded by // or enclosed by /*----------*/

d) Syntax: Grammar rules defined in a programming language.

Lecture 7: Lexical elements in C language:

List of keywords in C language

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Lecture 7: Lexical elements in C language:

Syntax rules in C language

Syntax – Formal set of conventions that determine the **structure** and **composition** of valid statements in a language.

- Use #include<stdio.h> header file.
- Each program has one **main** method.
- Never user undeclared variable.
- C is case sensitive
- A statement must end with a semi colon unless it's a pre-processor directive.
- Keywords must be in lowercase.
- C is a strongly typed language
- Do not place semicolon after function definition header.
- C program must be compiled before execution
- System headers use angle brackets < >.
- Code blocks are enclosed in curly braces { }.

Lecture 7: Lexical elements in C language:

Lexical analysis and Syntax analysis in C

Lexical analysis is the initial step in compiling a C program.

- It **scans the source code** and transforms it into a **series of tokens**, which are the smallest meaningful elements such as keywords, identifiers, literals, punctuators and operators.
- This process is handled by the **lexical analyzer**, also known as the **lexer.**
- Lexical analysis divides a C program into tokens, which are then sent to the next step of compilation, called **syntax analysis**

Lecture 7: Lexical elements in C language:

Steps in Lexical Analysis:

a) **Input**: The C program's source code, consisting of a sequence of characters.

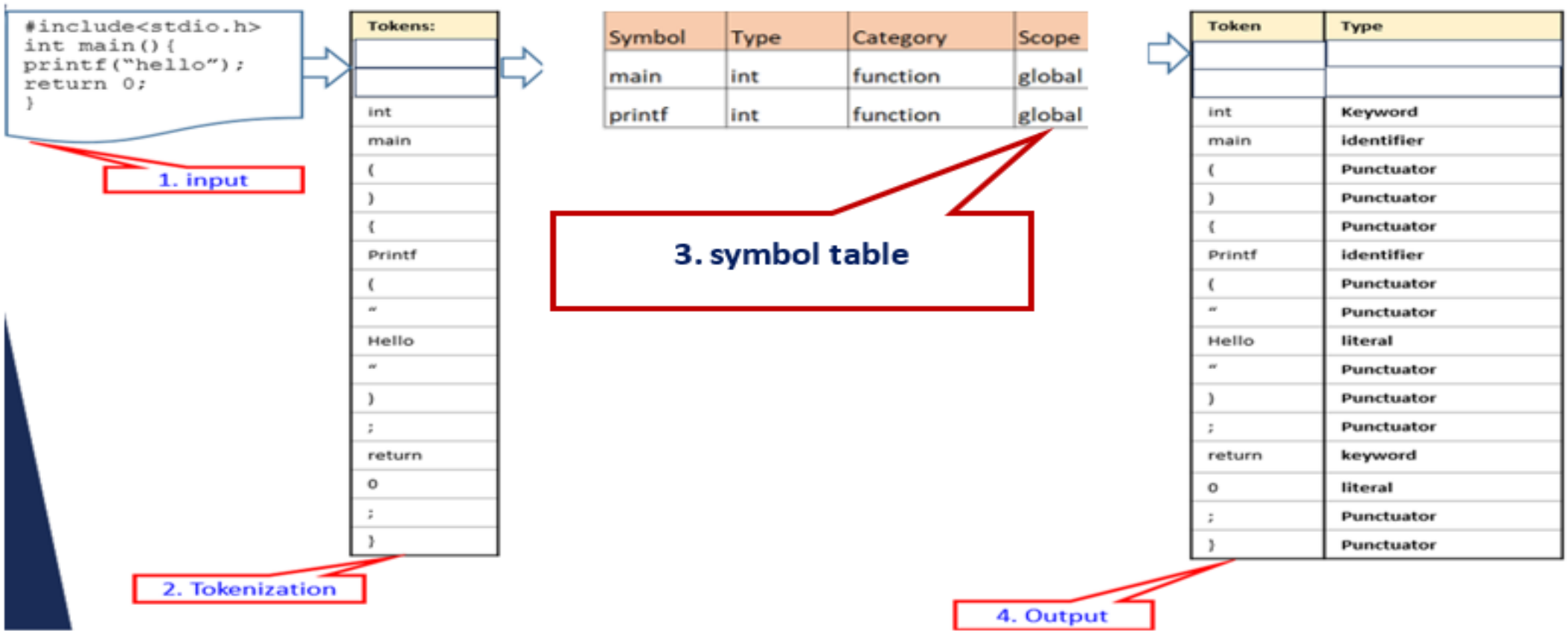
b) **Tokenization**: The process of converting the source code into **tokens**.

c) **Symbol table creation**: Recording identifiers (such as variable names) in a symbol table.

d) **Output**: A sequence of tokens that outlines the program's structure.

Lecture 7: Lexical elements in C language:

Steps in Lexical Analysis:



Lecture 7: Lexical elements in C language:

Syntax Analysis in C

Syntax analysis is the **second step** in compiling a C program, after lexical analysis.

The syntax analyzer, also called the parser, checks whether the tokens follow the grammar rules of the C language.

Steps involved:

- The parser receives tokens from the lexical analyzer.
- It verifies that the tokens are arranged according to C grammar rules.
- It organizes them into meaningful structures such as expressions, statements, and blocks.
- It builds a **syntax tree** that represents the program structure.
- It reports syntax errors if any rules are violated.

NB: A syntax tree visually shows how the compiler understands the structure of a program.

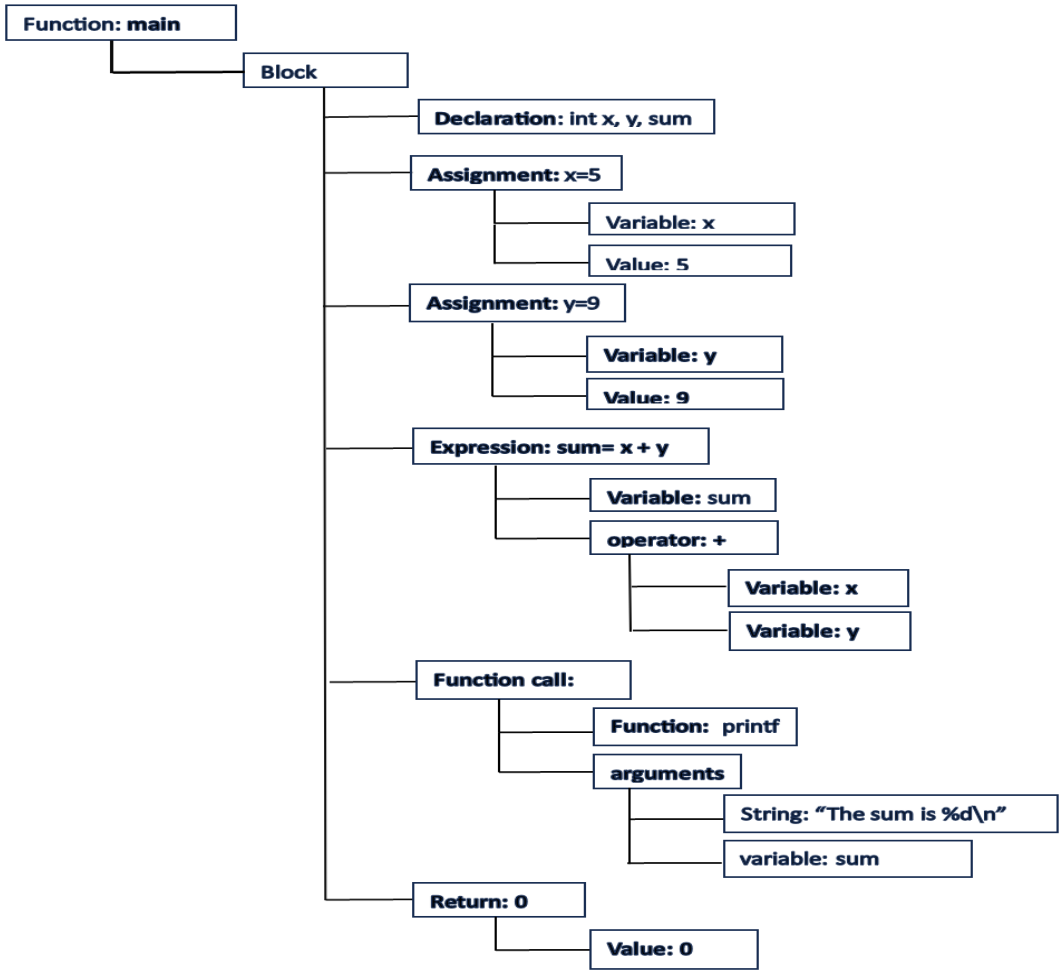
Lecture 7: Lexical elements in C language:

Example: Given the C program below, draw the abstract syntax tree (AST) showing declarations, assignments, expressions, function calls, and the return statement, where applicable.

```c
#include<stdio.h>
int main(){
int x,y,sum;
x=5;
y=9;
sum=x+y;
printf("The sum is %d\n",sum);
return 0;
}
```

Lecture 7: Lexical elements in C language:

Syntax Analysis in C



Lecture 7: Lexical elements in C language:

Activity 7.

a) Describe the role of lexical analyzer

b) Explain the output of the lexical analyzer for the code below

```
#include<stdio.h>
 int main()
 {
    int a,b;
    a=90;
    b=70;
    if(a<b){
      printf("a is greater\n");
     }
    return 0;
  }
```

Lecture 7: Lexical elements in C language:

Exercise.

a) Describe the steps followed by a lexical analyzer
b) Explain the process of syntax analysis in C
c) Describe any four tokens in C language.
d) Explain two ways of commenting the source code in C.

Lecture 7: Lexical elements in C language:

Summary

- In this lecture we learn that lexical analysis is the first phase of a C compiler, where raw source code is broken down into meaningful units called tokens.
- It categorizes elements like keywords, identifiers, and operators, while stripping away whitespace and comments.
- During this process, the analyzer builds a foundational symbol table to track user-defined variables and functions.
- This clean stream of tokens is then passed forward for syntax checking.

Lecture 7: Reference:

1. An introduction to programming by Wu Thomas, Norman and Theodore: McGrill (1997).
2. C programming language by Brian W. Kernighan, Dennis Ritchie, 2nd ed: prentice hall(2003).
3. C how to program by H.M. Deitel and P.J.Deitel, 3rd ed: Prentice hall(2001).
4. Su, Y., & Yan, S. Y. (2011). Lexical Analysis. In *Principles of Compilers: A New Approach to Compilers Including the Algebraic Method* (pp. 79-99). Berlin, Heidelberg: Springer Berlin Heidelberg.